GEMM Engine v1

Microarch Document

Version 1.3 · 2026-10-30

Aayush Soni

Contents

Document information 5
Conventions 5
1. Overview 6
2. Top-level architecture 8
2.1 Block diagram 8
2.2 Top-level ports 10
2.3 Parameters and constants 10
2.4 Pass pipeline timing 10
2.5 Clock and reset 13
2.6 System expectations 13
2.7 Operation 13
3. Interfaces 15
3.1 Protocol conventions 15
3.2 External interfaces 15
3.3 Internal command buses 17
3.4 Point-to-point signals 18
4. Submodules 20
4.1 Top controller (TC) 20
4.2 Weight controller (WC) 21
4.3 Activation controller (AC) 23
4.4 Output controller (OC) 25
4.5 Accumulator datapath 27
4.6 Systolic array and PE 27
4.7 Skew, deskew and delay lines 28
4.8 Infrastructure 28
5. Memories and storage 29
5.1 SRAM instances 29
5.2 FIFOs 29
5.3 Flop arrays 29
6. Performance 31
7. Verification summary 31
8. Implementation results 31
8.1 Synthesis 31
8.2 Place and route 32
8.3 Power 33
9. Limitations and future work 34
Appendix A. Glossary 35

Figures

Figure 1: Tiling of C = A · B: the pass (r, n, k) multiplies A_rk by B_kn and accumulates into C_rn. 6
Figure 2: Datapath of gemm_top 8
Figure 3: Control plane of gemm_top 9
Figure 4: One pass at full rate; faded bars are the next pass 12
Figure 5: Abort at c = 16 with AC ready (left) and at c = 17 with AC not ready (right); dashed red boxes are pulses the abort removes 14
Figure 6: Output streams across a block boundary: block b+1's header arrives while c1 still carries block b 16
Figure 7: WC - AC interlock at full rate: flag, go and the next load all at c = 16 18
Figure 8: TC state machine 20
Figure 9: Weight controller high level diagram 22
Figure 10: Activation controller high level diagram 24
Figure 11: Output controller high level diagram 26
Figure 12: Accumulator datapath, one bank 27
Figure 13: PE weight-buffer states 27
Figure 14: Skew and deskew triangles (one register per square, one lane wide) 29

Tables

Table 1: Revision history 5
Table 2: References 5
Table 3: v1 at a glance 7
Table 4: Module hierarchy 9
Table 5: gemm_top ports (DV / DVS buses listed once; the handshake is implied) 10
Table 6: gemm_v1 constants 10
Table 7: Key cycles of a pass 11
Table 8: Edge-tile handling 14
Table 9: cmd_t, 52 b 15
Table 10: resp_t, 8 b 15
Table 11: Input streams 16
Table 12: C output streams 16
Table 13: c_hdr_t, 16 b 16
Table 14: wcmd_t, 11 b 17
Table 15: acmd_t, 26 b 17
Table 16: Interlock signals 18
Table 17: Array control signals 19
Table 18: Datapath control signals 19
Table 19: TC states 20
Table 20: PE weight-buffer states 28
Table 21: SRAM instances (all ram_1r1w: 1 read + 1 write port, 1-cycle synchronous read) 29
Table 22: Command and ID FIFOs (sync_fifo) 29
Table 23: Large flop arrays 29
Table 24: Measured on the RTL (16.00 cycles per pass in every case) 31
Table 25: Synthesis results (flat netlist, pre-layout) 31
Table 26: Cell area by block (hierarchical synthesis) 32
Table 27: Place-and-route results, Nangate45 typical corner 32
Table 28: Power at 500 MHz by workload 33
Table 29: Power by block, filter-bank workload (logic counted with the block of the flop it feeds) 33

Document information

Table 1: Revision history

Version	Date	Change
1.1	2026-10-29	First draft
1.2	2026-10-29	Synthesis results (pre-layout, Nangate45)
1.3	2026-10-30	Place-and-route and power results

Table 2: References

Reference	What
`docs/spec.md`	v1 specification: requirements, interfaces, decisions and their reasons
`prj/v1/*.sv`, `rtl/*.sv`	RTL this document describes

Conventions

- **Bus naming:** A DVS bus is `<bus>_data` / `<bus>_valid` / `<bus>_stall`; a DV bus has no stall. Buses are described in Sec 3; module sections point there.
- **Cycle notation:** *c* counts cycles from the start of a weight load (c = 0 is the cycle the load's first read is issued). *G* is the cycle of a pass's **go** (AC starts the pass); at full rate G = c 16 of the pass's weight load.
- **Constants** are named as in `gemm_v1_pkg` (`T` = 16 lanes, `S` = 8 lanes per bank, `R` = 16 rows per pass, …; Sec 2.3).
- **Banks:** Every buffer is split by width into bank 0 (lanes 0–7) and bank 1 (lanes 8–15). Bank 1 is always accessed S = 8 cycles after bank 0.
- **Figures:** SRAMs are square-cornered; logic and registers are rounded; muxes and adders are drawn as symbols.

1. Overview

The GEMM engine computes **C = A · B** for signed int8 matrices of any shape (M, K, N from 1 to 65 536) with exact int32 results. Throughout this document the size conventions are A is M × K, B is K × N, and C is M × N. v1 is built around one 16 × 16 weight-stationary systolic array: B is tiled into 16 × 16 weight tiles, each held in the array for one **pass** while 16 rows of A stream through it, and k is the innermost loop so one 16 × 16 int32 accumulator holds the C block being built.

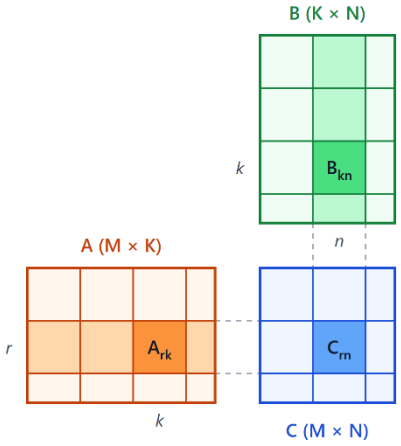


Figure 1: Tiling of C = A · B: the pass (r, n, k) multiplies A_rk by B_kn and accumulates into C_rn.

The matrix computation can be represented as `C[m][n] = Σ_k A[m][k] · B[k][n]`. The GEMM engine tiles A, B, C, effectively computing `C_rn = Σ_k A_rk · B_kn`, where X_ij represents the (i,j)th tile of matrix X, where each tile is a 16x16 chunk of the entire matrix. The weight-stationary implementation computes the innermost loop of the following algorithm, highlighted in yellow:

```
for r = 0 .. ⌈M/16⌉ − 1                  // block row of C
  for n = 0 .. ⌈N/16⌉ − 1                // block column of C
    for k = 0 .. ⌈K/16⌉ − 1              // one pass per k
      load weight tile B_kn into the array
      stream the rows of A_rk through it
      ACC  = (k == 0 ? 0 : ACC) + A_rk · B_kn   // 16 × 16 int32
    output ACC as C block C_rn
```

The host sends one descriptor per GEMM; A and B arrive as two data-only streams in pass order; C leaves as two half-row streams plus a block header. Four controllers sequence the work: the **top controller (TC)** walks the tile loops, the **weight (WC)** and **activation (AC)** controllers own the input banks and feed the array, and the **output controller (OC)** accumulates and outputs C.

A pass is one iteration of the inner loop and takes 16 cycles:

1. The weight tile is resident. B_kn is held in the array; PE (row j, column i) holds `B_kn[j][i]`.
2. The A rows stream through. Each of the (up to 16) rows of `A_rk` enters the left edge. Row x carries `A_rk[x][0..15]`, one element per array row.
3. Each column produces one psum. As row x passes through column i, the column accumulates down its 16 PEs:

$$psum[x][i] = \Sigma_j\ A_rk[x][j] \cdot B_kn[j][i]$$

So one pass produces a 16 × 16 block of partial sums, `psum = A_rk · B_kn`: one psum row per A row, 16 columns each.

The v1 core targets int8 convolution and matrix workloads at 500 MHz (128 G MAC/s peak): for example, a bank of 16 3 × 3 filters over grayscale video at 1080p and 240 fps, or 4K at 60 fps. The same throughput runs a ResNet-18-class CNN at roughly 60 images per second, assuming the input streams are fed from on-chip memory at full rate (note that this would require an input data stream of ~16GBps and output stream support of ~32GBps). The output stream size can be reduced by re-quantizing the values to int8, though v1 does not support it right now.

Table 3: v1 at a glance

Item	**v1**
Array/ Systolic Array	One `systolic` (M = 16): 256 PEs, int8 × int8 + 20-bit psum, double-buffered weight per PE
Pass	16 cycles: one B tile resident, 16 A rows streamed; the next tile loads during the pass
Throughput	One pass per 16 cycles at steady state; cycles ≈ 16 · (M/16)(N/16)(K/16) + 64.
Input banks	A and B: 2 banks × 32 rows × 64 b each (ring buffers)
Accumulator	2 banks × 16 words × 256 b = 1 KB
Clock / reset	Single clock; synchronous active-high reset; `abort_req` as a soft reset of control state

2. Top-level architecture

2.1 Block diagram

`gemm_top` instantiates the array, the six input and accumulator banks, the skew and deskew triangles and the four controllers. The following figures show the datapath and control plane.

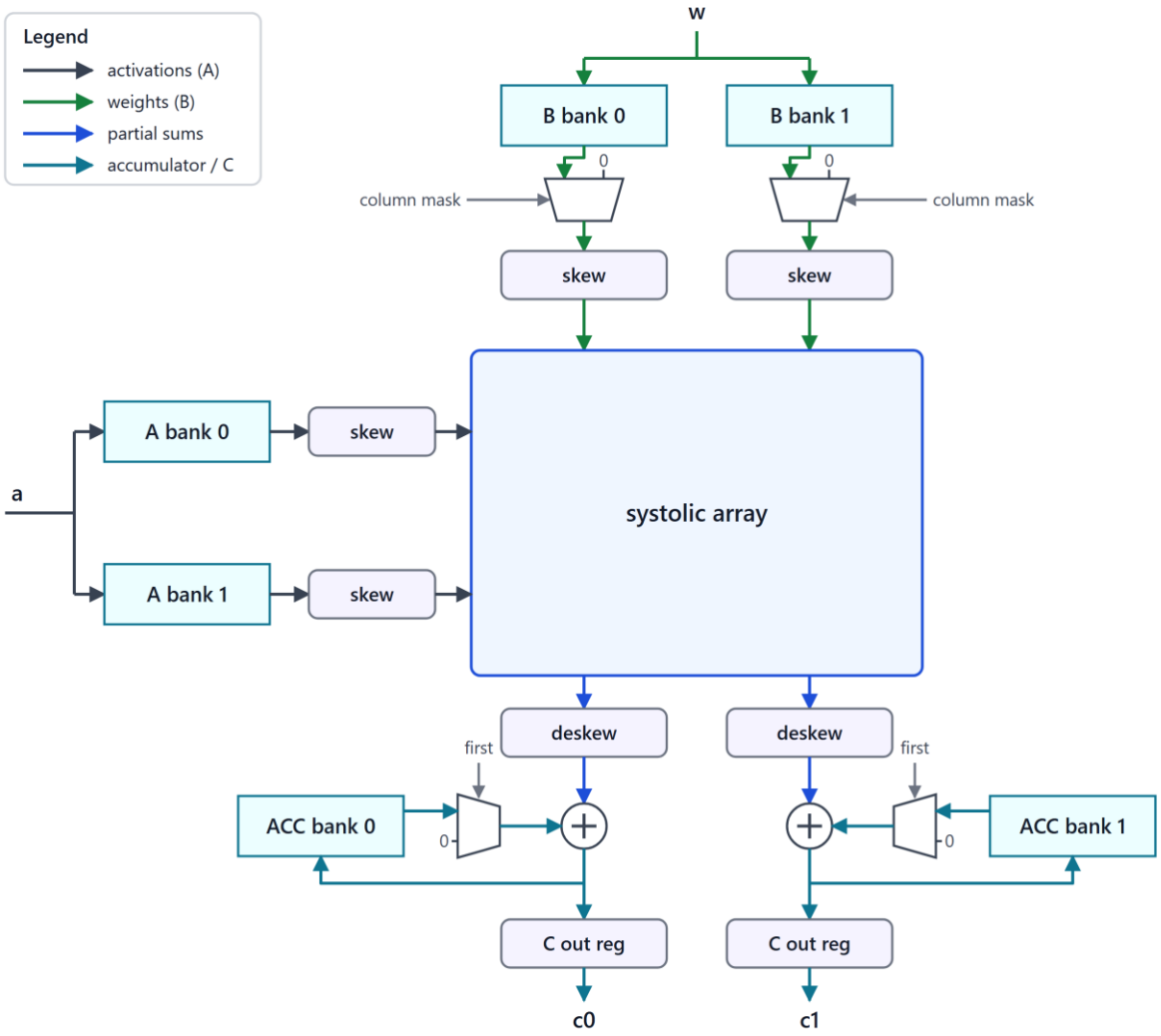


Figure 2: Datapath of gemm_top

B rows are written to both B banks at the same address; bank 1 is read 8 cycles after bank 0, so each half needs only a 0→7 skew triangle (lanes 8–15 get their extra 8 cycles from read timing). Before the skew, a mux per bank forces *padding* rows and columns to 0. "A" rows take the same path into the array's left edge. Psums leave the bottom of each column, are realigned by a 7→0 deskew per half and added to the accumulator word (or to 0 on a block's first pass); the sum is written back, or on a block's last pass loaded into the C out register.

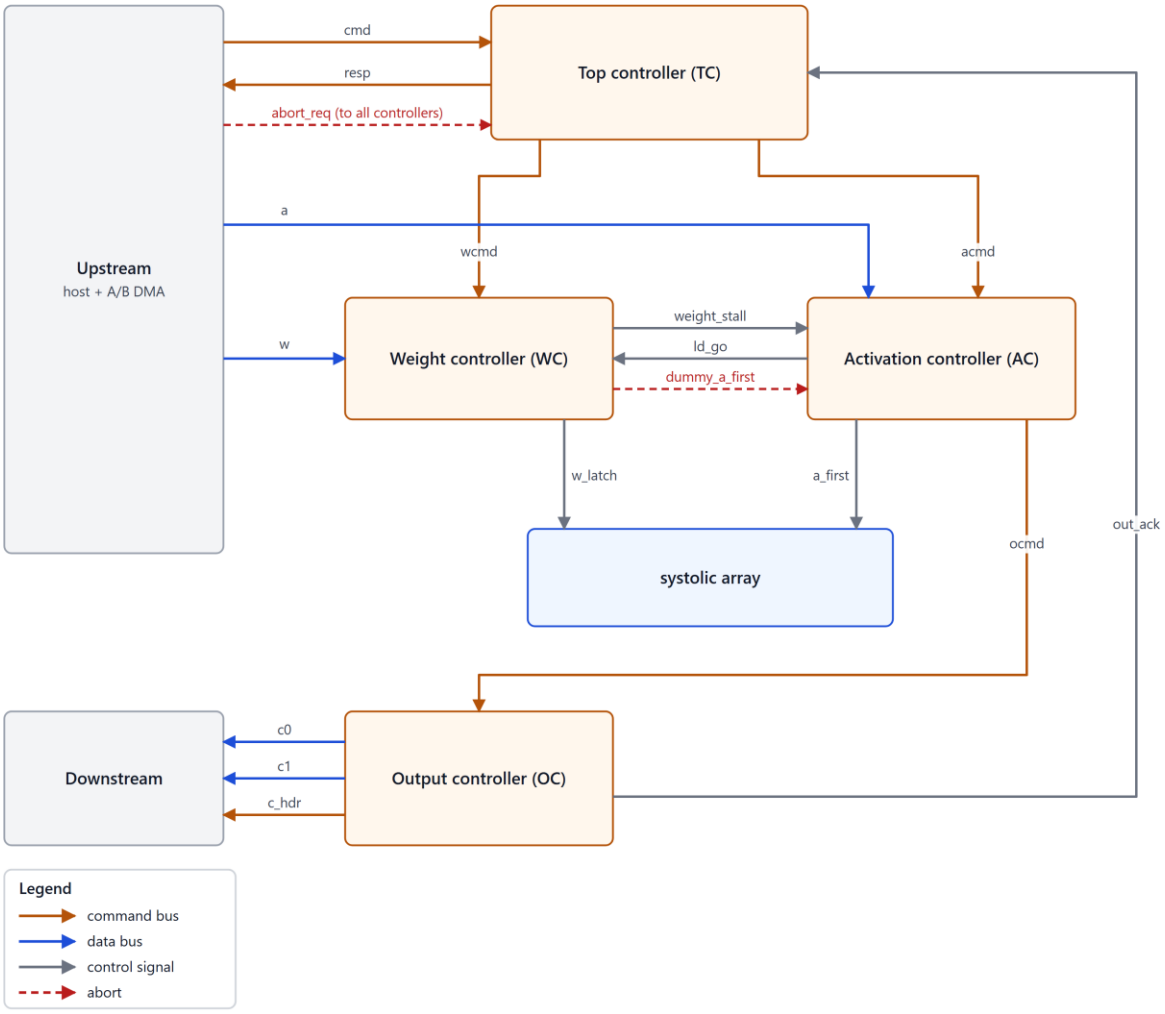


Figure 3: Control plane of gemm_top

TC turns the input descriptor into one pass command per pass, sent to WC (`wcmd`) and AC (`acmd`) together. WC and AC each own their bank's pointers and decide on their own when the next tile is complete; they meet through the WC ↔ AC interlock. AC forwards the output fields of each pass command to OC (`ocmd`) when the pass actually starts, and OC reports each finished GEMM to TC (`out_ack`).

Table 4: Module hierarchy

Module	File	Role
`gemm_top`	`prj/v1/gemm_top.sv`	Top level wrapper: wiring, banks, skews, accumulator datapath
`top_controller`	`prj/v1/top_controller.sv`	Descriptor intake, tile loops, pass commands, responses, abort
`weight_controller`	`prj/v1/weight_controller.sv`	B banks, weight loads, masks, `w_latch`, interlock
`activation_controller`	`prj/v1/activation_controller.sv`	A banks, passes, `a_first`, `ocmd`
`out_controller`	`prj/v1/out_controller.sv`	Accumulator control, C out valids, header, `out_ack`
`systolic`, `pe`	`rtl/systolic.sv`, `rtl/pe.sv`	16 × 16 array of PEs
`skew`, `delay_line`	`rtl/skew.sv`, `rtl/delay_line.sv`	Skew / deskew triangles

Module	File	Role
ram_1r1w, sync_fifo	rtl/ram_1r1w.sv, rtl/sync_fifo.sv	SRAM wrapper and control FIFOs
gemm_v1_pkg	prj/v1/gemm_v1_pkg.sv	Constants, bus structs, timing constants

2.2 Top-level ports

Table 5: gemm_top ports (DV / DVS buses listed once; the handshake is implied)

Bus	Dir	Payload	Sec
cmd	DVS in	cmd_t {ID, M, K, N}, 52 b	3.2.1
resp	DV out	resp_t {ID, done, info}, 8 b	3.2.2
abort_req	in	one-cycle pulse	3.2.3
w	DVS in	one B row, 16 × int8	3.2.4
a	DVS in	one A row, 16 × int8	3.2.4
c0, c1	DV out	C columns 0–7 / 8–15 of one row, 8 × int32 each	3.2.5
c_hdr	DV out	c_hdr_t block header, 16 b	3.2.6

2.3 Parameters and constants

All constants live in gemm_v1_pkg.

Table 6: gemm_v1 constants

Constant	Value	Meaning
IN_W, ACC_W	8, 32	A/B element width; accumulator / C element width
T, S, GRID	16, 8, 2	Array size; lanes per bank (half-width); banks per side
R, ACC_DEPTH, ACC_AW	16, 16, 4	Rows per pass; accumulator words per bank; its address width
W_DEPTH, A_DEPTH (W_AW, A_AW)	32 (5)	B / A bank rows. Must be a power of 2 (pointers wrap with one extra bit)
PSUM_W	20	Exact psum width for a 16-deep column: 2·IN_W + log2(16)
DIM_W, SIZE_W, TAG_W, ID_W	16, 4, 3, 4	Descriptor field; tile size (0 = 16); pass tag; GEMM ID
OC_PSUM_LAT	24	go → half-0 row 0 at the OC input: 1 (a_first reg) + T + (S – 1) deskew
OC_RD_START, OC_POP	23, 38	go → bank 0's first ACC read; its last read (ocmd FIFO pop)
WCMD_W, ACMD_W, OCMD_W, C_HDR_W	11, 26, 22, 16	Bus struct widths

2.4 Pass pipeline timing

Figure 4 follows one pass at full rate. The weight load for pass p starts at c = 0; pass p itself starts (AC's go) at c = 16, the same cycle the load for pass p+1 starts. From the go, every later step is a fixed offset.

Table 7: Key cycles of a pass

Event	Cycle	Source
Bank 0 weight reads (bank 1: +8)	c 0–15 (8–23)	WC load counter
`weight_stall` visible, AC go, next load starts	c 16	WC flag, AC go
`w_latch` at the array (column 0)	c 17	WC register
A reads, bank 0 (bank 1: +8)	G + 0–15 (G + 8–23)	AC pass counter
`a_first` lane j at the array	G + 1 + j	AC shift register
psum row x, column i leaves the array	G + 17 + x + i	array
OC bank 0 ACC read (bank 1: +8)	G + 23 + x	OC read stage
`c0` row x valid (`c1`: +8); header with row 0	G + 25 + x	OC output registers
`out_ack` (last pass of a GEMM)	G + 33 + (m − 1)	OC bank 1 timeline

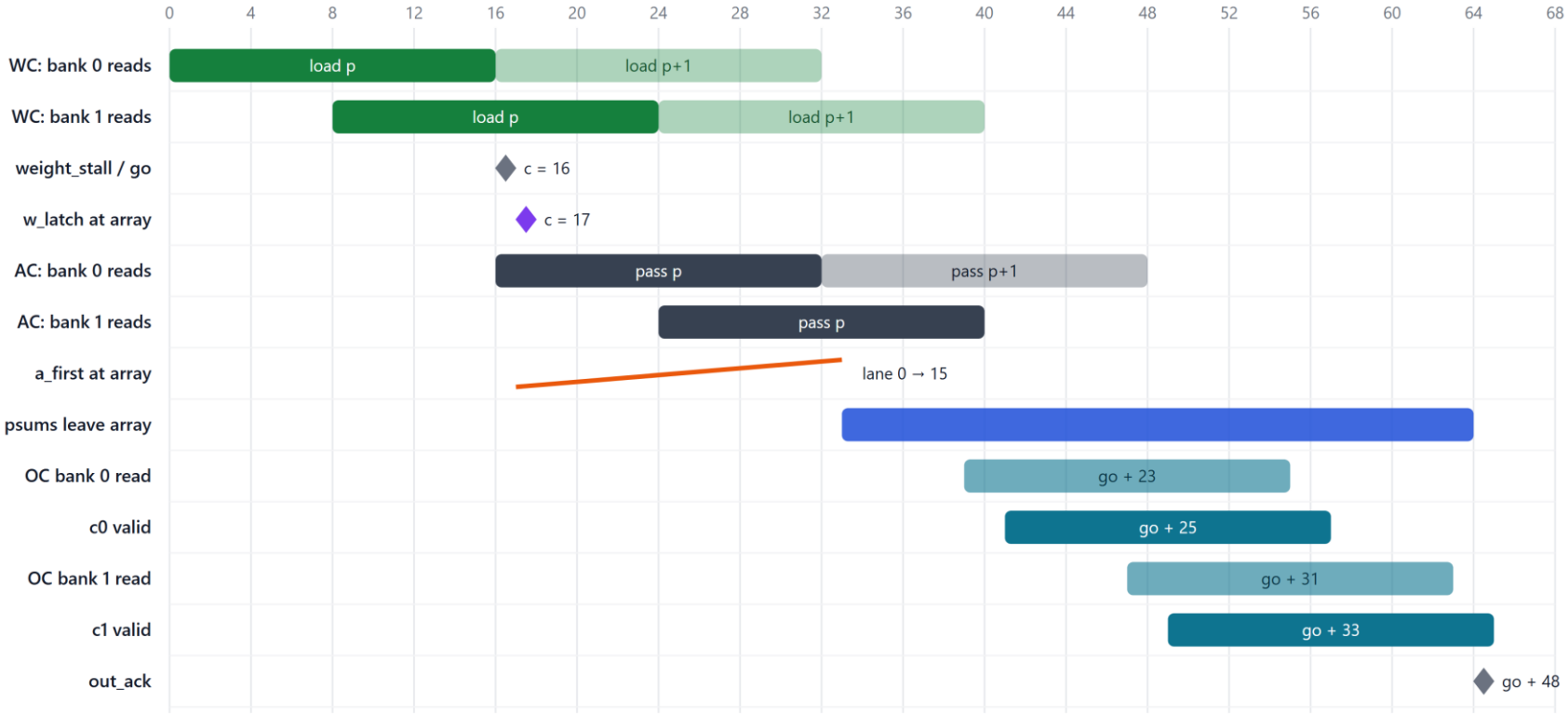


Figure 4: One pass at full rate; faded bars are the next pass

2.5 Clock and reset

- Single clock domain (`clk`); design uses rising edge sampling.
- `rst` is synchronous and active high. It clears all control state, the PE flags and the skew registers; SRAM contents are not reset.
- `abort_req` is a soft reset of control state only. The array, skews and SRAMs keep running; nothing reads stale data without pointers.

2.6 System expectations

Host

- Sends one descriptor per GEMM on `cmd` and reads `resp`: each accepted descriptor gets one accept and one end response, in order.
- Up to 4 GEMMs can be outstanding (TC's pending FIFO); `cmd` stalls beyond that.
- Keeps the IDs of outstanding GEMMs unique if it matches blocks or responses by ID (GEMMs finish in order, so order is a fallback).
- Programs upstream with the same shapes as the descriptors.
- Abort is a one-cycle pulse on `abort_req`; every outstanding GEMM then ends as aborted.

Upstream (A and B streams)

- Sends tiles in the loop order: for each (r, n), A tiles A_r0 … A_r,K−1 and B tiles B_0n … B_K−1,n; A is re-sent for every n and B for every r.
- Within a tile: A rows in M order; **B rows in reverse K order** (row k-size − 1 first).
- Sends only real rows (k-size B rows, m-size A rows per tile); lanes outside the tile are don't-care.
- May send rows before their GEMM is accepted and may pause between beats; the banks absorb both.
- After an abort, discards every beat of the aborted GEMMs it still holds and sends only the next GEMM's data (stale-beat contract). `w_stall` / `a_stall` stay high until the next GEMM's first pass command, so beats in flight are not taken.

Downstream (C and header)

- Must accept every `c0`, `c1` and `c_hdr` beat in the cycle it is valid (no stall signal).
- Uses the block header (one per block, with its first `c0` row) and the output contract (described in Sec 3.2.5–3.2.6): fixed `c1` = `c0` + 8 timing, m-size rows per block, lanes ≥ n-size are 0.
- After an abort, treats the next header with `r_first && n_first` as the start of a new GEMM; the aborted GEMM's last block may be cut short.

2.7 Operation

2.7.1 Normal pass sequence

1. TC accepts a descriptor (accept response) and issues one pass command per pass to WC and AC, as fast as their 4-entry FIFOs take them.
2. WC writes B rows into its banks as they arrive. When the head command's tile is complete (and the interlock allows), it starts a 16-cycle load: zero rows first for a short K tile, then the tile's rows, bank 1 eight cycles behind.
3. At c = 16 WC raises `weight_stall`; AC, holding the matching A rows, starts the pass in the same cycle (go), which also lets WC start the next load.
4. `w_latch` and `a_first` reach PE[0][0] at c = 17: the PEs latch the new tile and swap to it as the pass's first A row arrives.

5. AC forwards the pass's output fields to OC at the go. OC accumulates the 16 psum rows into the ACC; on the block's last pass it outputs them on `c0` / `c1` with a header on row 0.
6. On the GEMM's last pass OC raises `out_ack`; TC sends the end response.

2.7.2 Back-to-back GEMMs

WC and AC see only a stream of pass commands, not GEMM boundaries, and TC accepts the next descriptor as soon as it has issued the current GEMM's last pass. So the next GEMM's first weight load overlaps the current one's last pass and GEMMs run without a bubble: measured, the next GEMM's first go comes exactly 16 cycles after the previous GEMM's last. Fill and drain are paid once per run of GEMMs.

2.7.3 Edge tiles

Table 8: Edge-tile handling

Partial dim	Handled by	Action
K (k-size < 16)	WC	The load still takes 16 cycles: 16 − k-size zero rows first (no read, bank output forced to 0), then the k-size rows. They land in PE rows 0..k-size − 1. A lanes ≥ k-size meet zero weights.
M (m-size < 16)	AC, OC	AC reads the m-size rows first and idles for the rest of the 16-cycle pass; OC never outputs psum rows ≥ m-size.
N (n-size < 16)	WC, OC	WC's column mask zeroes weight columns ≥ n-size, so those C lanes are exactly 0. With n-size ≤ 8, `c1` is idle for the block.

2.7.4 Abort

A one-cycle `abort_req` reaches all four controllers.

1. TC moves to `ST_ABORT` and sends one aborted end response per outstanding ID, one per cycle; `cmd` stalls meanwhile.
2. WC and AC clear their pointers, command FIFOs and counters; WC clears its interlock flag and pending `w_latch`, AC drops a go in the abort cycle. Both hold their input stall high until the next GEMM's first pass command.
3. OC clears its FIFO, start delay and pipeline, so no further `c0` / `c1` / `c_hdr` valid or `out_ack` appears for aborted passes.
4. The array is not reset, so its weight double-buffers must end consistent. Two rules ensure it: `a_first` diagonals already in flight are kept, and if a tile's `w_latch` reached the array but its pass never started, WC raises `dummy_a_first` and AC sends one `a_first` diagonal with no data (Figure 5).

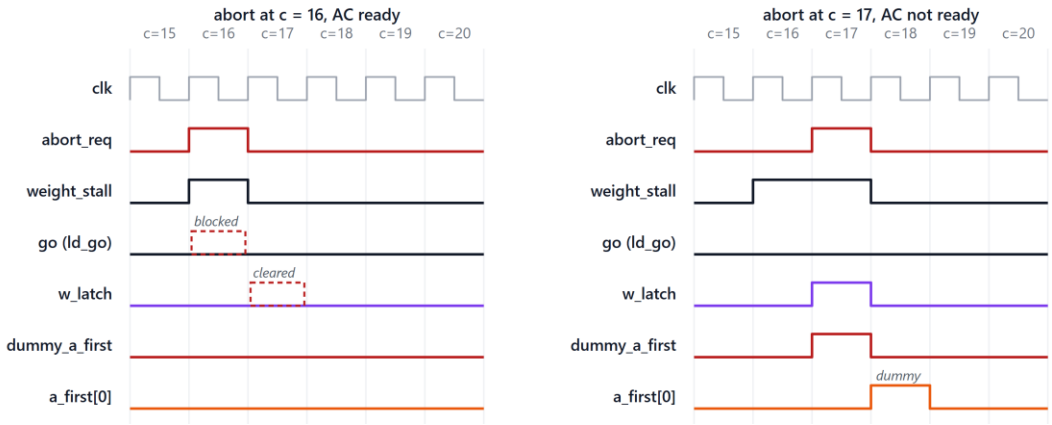


Figure 5: Abort at c = 16 with AC ready (left) and at c = 17 with AC not ready (right); dashed red boxes are pulses the abort removes

3. Interfaces

3.1 Protocol conventions

- **DVS** (data / valid / stall): a beat transfers in a cycle where `valid` is high and `stall` is low, and only then.
- Sinks never assume `valid` is held: a beat that is withdrawn before it transfers simply did not happen. External sources should hold `valid` and the data until the transfer (AXI-style); internally, the shared command stall may withdraw `valid`.
- Every stall comes from registered state; no stall depends on the `valid` presented in the same cycle.
- **DV** (data / valid): no backpressure; the sink takes every valid beat.
- Size fields are 4 bits with 0 = 16; descriptor dimensions are 16 bits with 0 = 65 536.
- Structs are packed with the first field in the MSBs.

3.2 External interfaces

3.2.1 cmd (DVS in): GEMM descriptor

Table 9: cmd_t, 52 b

Field	Bits	Meaning
`id`	51:48	GEMM descriptor ID
`m`	47:32	Rows of A and C (elements; 0 = 65 536)
`k`	31:16	Columns of A / rows of B
`n`	15:0	Columns of B and C

TC takes a descriptor only when it has issued every pass of the current GEMM and fewer than 4 GEMMs are outstanding. A descriptor accepted in the abort cycle is aborted too.

3.2.2 resp (DV out): responses

Table 10: resp_t, 8 b

Field	Bits	Meaning
`id`	7:4	Descriptor ID
`done`	3	0 = accept, 1 = end
`info`	2:0	Accept: warning bits, bit 0/1/2 = M/K/N field was 0. End: status, 0 = OK, 3 = aborted (1, 2 reserved)

Per accepted descriptor: one accept when it is taken, one end when its last C row has left (`out_ack`) or when it is aborted, all in command order. At most one accept and one end are produced per cycle; a collision delays the end by one cycle (one holding register in TC).

3.2.3 abort_req (in)

One-cycle pulse. Drops all pending work. TC sends an abort end response for every pending descriptor req.

3.2.4 w and a (DVS in): input streams

Table 11: Input streams

Bus	Data	Stall high when
w	w_data[15:0] × int8: one B row (lane = N column)	all 32 B bank slots hold rows bank 1 still needs; or from an abort until the next GEMM's first wcmd
a	a_data[15:0] × int8: one A row (lane = K column)	all 32 A bank slots hold rows bank 1 still needs; or from an abort until the next GEMM's first acmd

3.2.5 c0, c1 (DV out): C half-rows

Table 12: C output streams

Bus	Data	Timing
c0_out, c0_valid	C columns 0–7 of one row, 8 × int32	row x of a block at G + 25 + x (G = go of the block's last pass)
c1_out, c1_valid	C columns 8–15, 8 × int32	exactly 8 cycles after c0 row x; only if the block's n-size > 8

- Each active half sends exactly m-size rows per block, on consecutive cycles, in the loop order.
- Lanes ≥ n-size are 0 (c0: lanes ≥ min(n, 8); c1: lanes ≥ n − 8).
- A c1 row can arrive after the next block's header: blocks overlap by up to 8 cycles, and a short next block (even a whole short GEMM) can finish on c0 first (Figure 6). Downstream labels c1 rows by the fixed offset.
- The halves are never realigned in the engine (that would cost about 2k flops); each half can write its own downstream bank.

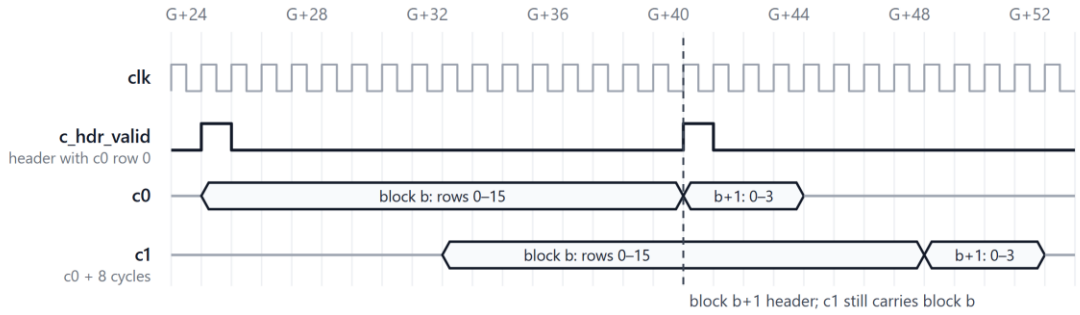


Figure 6: Output streams across a block boundary: block b+1's header arrives while c1 still carries block b

3.2.6 c_hdr (DV out): block header

Table 13: c_hdr_t, 16 b

Field	Bits	Meaning
r_first	15	First block row of the GEMM
r_last	14	Last block row of the GEMM
n_first	13	First block column
n_last	12	Last block column
m_size	11:8	Rows in this block (0 = 16)
n_size	7:4	Columns in this block (0 = 16)
id	3:0	Descriptor ID

Valid once per block, in the cycle of its first `c0` row; every block has one. A GEMM starts with the header that has `r_first && n_first` and ends with the last row of the block with `r_last && n_last`. Note: if the last block's n-size is <=8, the last c1 data will not appear (since cols 8-15 are not active) and so the last set of transactions will be c_hdr + c0 data and not c_hdr + c0 + c1.

3.3 Internal command buses

3.3.1 wcmd (DVS, TC → WC)

Table 14: wcmd_t, 11 b

Field	Bits	Meaning
k_size	10:7	Rows of this B tile (0 = 16)
n_size	6:3	Columns of this B tile
tag	2:0	Pass index mod 8 (for internal checking and debug)

3.3.2 acmd (DVS, TC → AC)

Table 15: acmd_t, 26 b

Field	Bits	Meaning
k_size	25:22	Number of columns of this A tile
first	21	k = 0
last	20	k = last of the block
gemm_last	19	Final pass of the GEMM (= `last && r_last && n_last`)
r_first	18	Block position (for the output header)
r_last	17	
n_first	16	
n_last	15	
m_size	14:11	Number of rows of this A tile; rows OC outputs
n_size	10:7	Columns OC outputs
id	6:3	Descriptor ID
tag	2:0	Pass index mod 8

3.3.3 ocmd (DV, AC → OC)

Field	Bits	Meaning
first	21	k = 0
last	20	k = last of the block
gemm_last	19	Final pass of the GEMM (= `last && r_last && n_last`)
r_first	18	Block position (for the output header)
r_last	17	
n_first	16	
n_last	15	
m_size	14:11	Number of rows of this A tile; rows OC outputs
n_size	10:7	Columns OC outputs
id	6:3	Descriptor ID

Field	Bits	Meaning
`tag`	2:0	Pass index mod 8

3.3.4 out_ack (OC → TC)

One-cycle pulse when a GEMM's last C row has left, generated on bank 1's timeline (G + 33 + m − 1 of the GEMM's last pass). TC ends its oldest outstanding GEMM; GEMMs finish in order, so the pulse carries no ID. Bank 1 runs every pass even when `c1` is idle, so pulses are in GEMM order and never coincide.

3.4 Point-to-point signals

3.4.1 WC - AC interlock

Table 16: Interlock signals

Signal	Dir	Meaning
`weight_stall`	WC → AC	A tile is being latched into the PEs' idle buffers and its pass has not started. Set so that it is visible at c = 16; cleared by `ld_go`
`ld_go`	AC → WC	AC's go this cycle: the pass for the latched tile starts, so WC may start the next load
`dummy_a_first`	WC → AC	Abort cycle only: a latched tile's pass never started; AC sends one dummy `a_first` diagonal.

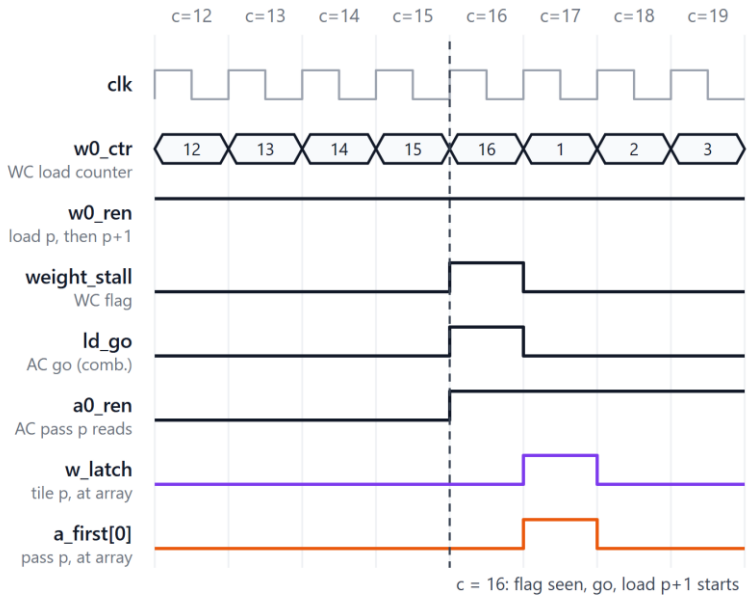


Figure 7: WC - AC interlock at full rate: flag, go and the next load all at c = 16

WC starts a load only when the shadow buffers are free: `!weight_stall || ld_go`. AC starts a pass only when `weight_stall` is high (the tile will be latched in time). The path `weight_stall` flop → AC go logic → WC start logic is combinational but has no loop.

3.4.2 Array control

Table 17: Array control signals

Signal	From	Meaning
`w_latch`	WC	Registered pulse at c = 17, broadcast to all 16 rows at column 0; the PEs carry it one column per cycle, matching the weight skew
`a_first[15:0]`	AC	Per-lane pulse marking a pass's first A row: lane j at G + 1 + j (a 16-stage shift register, so bank 1's lanes see the +8 offset too)

3.4.3 Bank RAM ports

Each `ram_1r1w` bank gets `we`, `waddr`, `re`, `raddr` from its controller; `wmask` is all ones. Banks 0 and 1 of A and B are written in the same cycle at the same address (one row per beat); bank 1 reads follow bank 0's by S cycles.

3.4.4 Datapath controls

Table 18: Datapath control signals

Signal	From	Meaning
`w0_zero`, `w1_zero`	WC	This cycle's B bank output is a padding row (K edge): force it to 0. Aligned with rdata; bank 1's is 8 cycles later
`w_col_mask[15:0]`	WC	Lane ≥ n-size of the tile being loaded: force that weight lane to 0. Latched per load (bank 1's copy 8 cycles later)
`c0_first`, `c1_first`	OC	Sum stage of a block's first pass: add 0 instead of the ACC read data
`c0_oen`, `c1_oen`	OC	Sum stage of an output row: load the C out register (`cX_valid` follows a cycle later)

4. Submodules

4.1 Top controller (TC)

TC accepts descriptors, walks the tile loops and issues one pass command per pass. It owns the response channel and the abort drain.

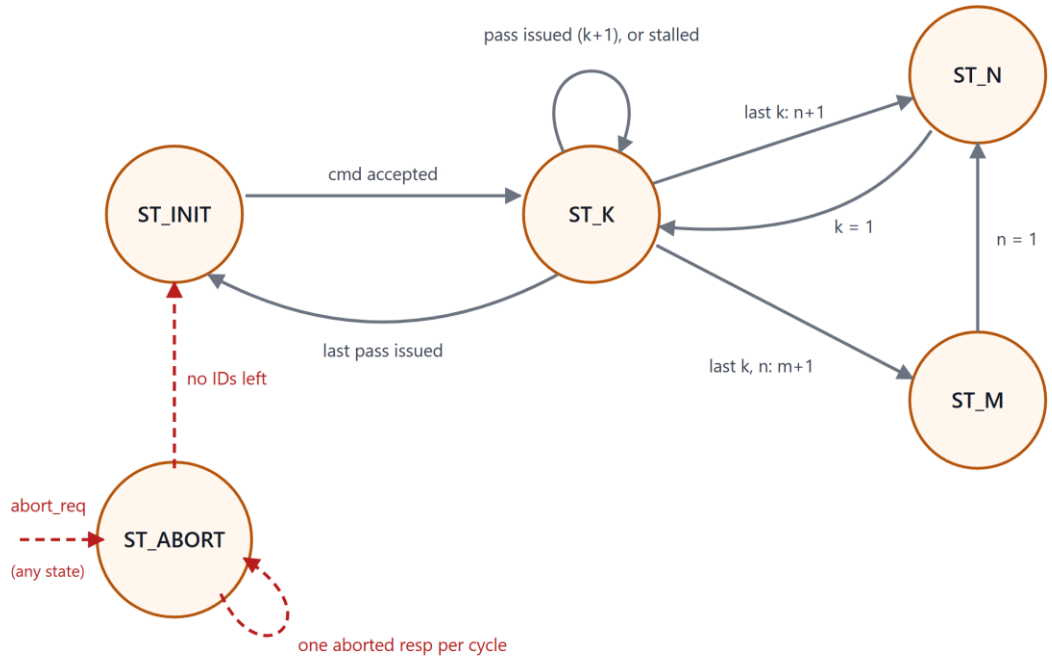


Figure 8: TC state machine

Table 19: TC states

State	Action
`ST_INIT`	Idle. On `cmd_valid` with room in the pending FIFO: latch the descriptor, set the k / n / m counters to 1, push the ID, send the accept response, go to `ST_K`
`ST_K`	Drive `wcmd` and `acmd` for the current (r, n, k). On a transfer: k + 1; at the end of the k loop go to `ST_N` (n + 1), at the end of the n loop to `ST_M` (m + 1), after the GEMM's last pass to `ST_INIT`. Stays while stalled
`ST_N`	One cycle: k = 1, back to `ST_K`
`ST_M`	One cycle: n = 1, to `ST_N`
`ST_ABORT`	One aborted end response per cycle for each ID in the pending FIFO, oldest first; `ST_INIT` when empty

- **Loop order:** for r (block rows), for n (block columns), for k: k innermost so a block finishes before the next starts.
- **Tile sizes:** 16, or the dimension's low 4 bits on its last tile (both encoded as 4 bits with 0 = 16). `first` = (k counter = 1), `last` / `r_last` / `n_last` = the loop-end flags.
- **Pending FIFO** (`sync_fifo`, 4 × 4 b): IDs of outstanding GEMMs, pushed at accept, popped at the end response.
- **Responses:** a registered `resp` output; an `out_ack` that collides with an accept is held one cycle (`ack_hold`). Ends are at least a cycle apart, so one holding register is enough.

- **Issue rate:** one pass command per cycle in `ST_K`; the one-cycle `ST_N` / `ST_M` steps are hidden by the units' 4-entry FIFOs.

4.2 Weight controller (WC)

WC owns the two B banks: it writes the B stream into them, loads one tile per pass command into the PEs' idle weight buffers, and drives `w_latch`, the padding masks and its side of the interlock.

Write side

- Pointers carry one wrap bit (6 bits for 32 rows): `wptr` (rows written), `rptr0` / `rptr1` (rows read by bank 0 / bank 1).
- A row is written whenever a slot is free: occupancy = `wptr − rptr1`, counted from bank 1 because bank 1 reads each row last. `w_stall` = occupancy == `W_DEPTH`, or the abort hold.
- Both banks are written at `wptr` in the same cycle.

Commands and load start

- `wcmd` FIFO stores 4 outstanding requests.
- To begin loading data into the systolic array:
  - A command is present (FIFO not empty)
  - Enough rows written into banks, i.e. (`wptr − rptr0`) ≥ k-size
  - The load counter is idle or at 16 (prev load is completed).
  - The shadow buffers are free.
- The load counter runs 1..16; a new load can start from 16, so loads are back-to-back.

Read side and masks

- Bank 0 reads in the last k-size cycles of the load; the first 16 − k-size cycles are padding rows.
- Bank 1's read enable and zero flag are bank 0's delayed by S = 8 cycles (`delay S`).
- The tile's n-size is latched at the load start (bank 0's mask) and copied 8 cycles later (bank 1's), giving `w_col_mask`.

Interlock, w_latch, abort

- `weight_stall` is set when the load reaches its last read (visible at c = 16) and cleared by `ld_go`.
- `w_latch` is registered from `w0_ctr == T`, reaching the array at c = 17.
- Abort clears pointers, FIFO, counter, delay line, flag and `w_latch`. `dummy_a_first` = abort ∧ `weight_stall` ∧ (`w0_ctr` != T): in the load's c = 16 the flag is set but `w_latch` is still in its register (and is cleared), so no dummy is needed. `w_stall` is held from the abort until the next `wcmd`.

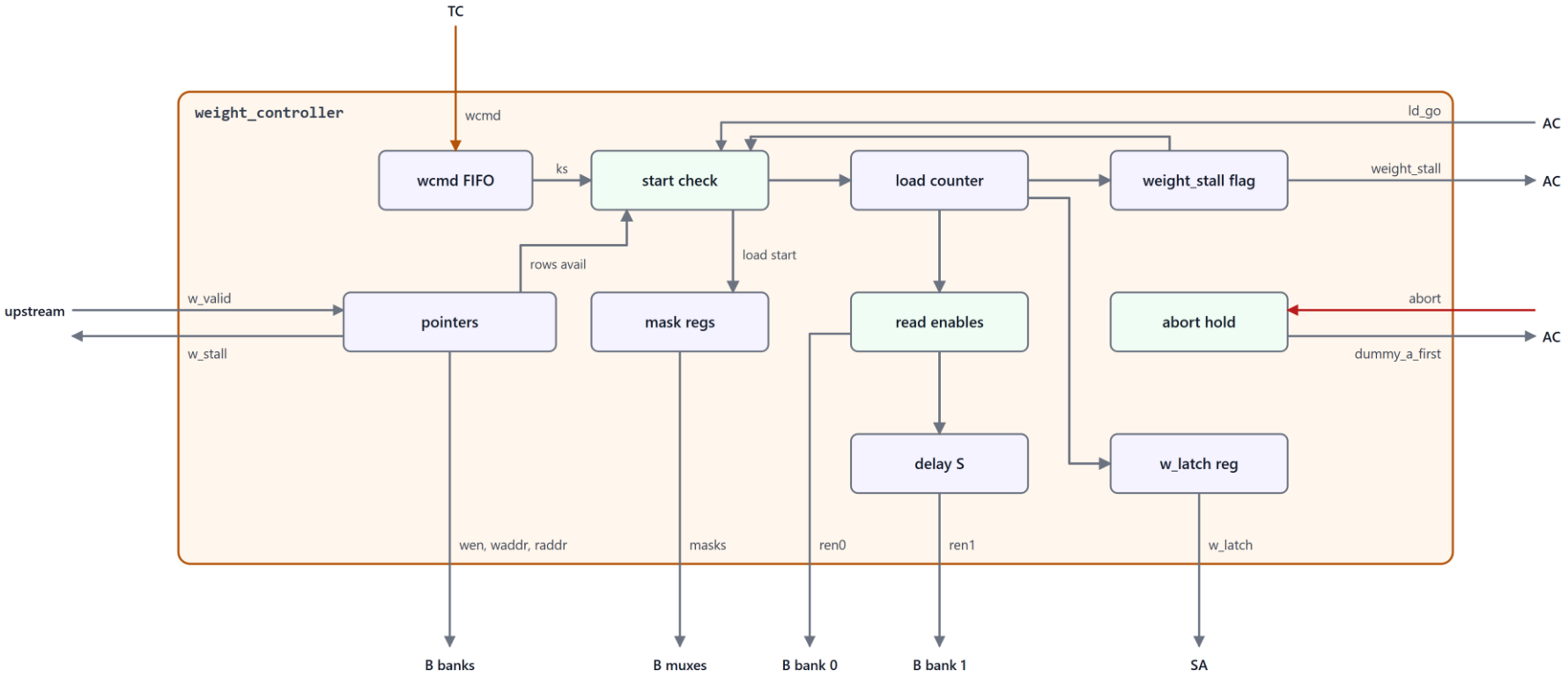


Figure 9: Weight controller high level diagram

4.3 Activation controller (AC)

AC mirrors WC for the A banks and starts each pass (the go). Figure 10 shows its structure.

- Same write side as WC (per-row gating on banks 0/1, occupancy from bank 1, `a_stall` with the abort hold).
- `acmd` FIFO, 4 × 26 b. AC starts each pass 16 cycles after WC starts the weight load.
- **Go:** `weight_stall` ∧ head present ∧ (`wptr − rptr0` ≥ m-size) ∧ pass counter idle or at 16 ∧ !`abort_req`. The go is `ld_go` to WC and `ocmd_valid` to OC, and enters the `a_first` shift register.
- **Reads:** bank 0 reads the m-size real rows first (`a0_ctr_next ≤ m-size`), then idles for the rest of the pass; bank 1 is bank 0 delayed by S. “A” lanes ≥ k-size need no masking since they meet the zero weight rows WC loads. Rows past m-size are not sent, and OC does not output them.
- **`a_first`:** a 16-bit shift register fed by the go (or `dummy_a_first`); lane j is j cycles behind lane 0. It is **not** cleared on abort, so a diagonal in flight completes and the dummy (entered in the abort cycle) is delivered.
- **`ocmd`:** the head's low 22 bits, valid with the go. Reason -> adding ocmd txfer to TC would mean extra FIFO depth needed (same cmd is stuck in 3 places across 3 different time segments) and needs extra wiring back to TC. Now AC can decide when to write the ocmd_data.

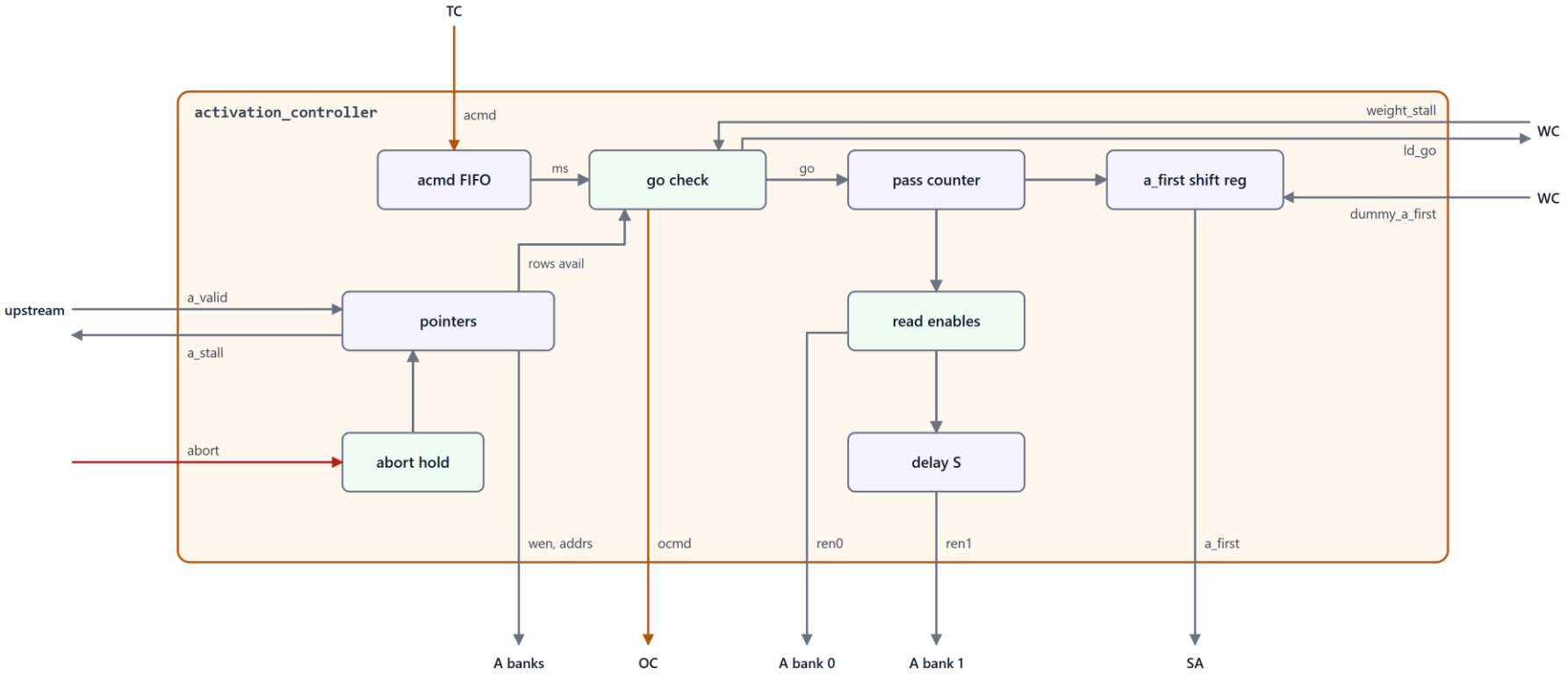


Figure 10: Activation controller high level diagram

4.4 Output controller (OC)

OC is timed entirely from AC's go (cycle G). Each bank has a **read stage** (row x: ACC read and decode of the pass fields) and a **sum stage** one cycle later (psum + ACC data: write-back or output). Everything the sum stage needs is decoded in the read stage and registered.

- **ocmd FIFO**, 4 × 22 b: pushed at each go; the head is bank 0's active pass and is popped at its last read (`OC_POP` = G + 38). Up to 3 entries are live (each lives 38 cycles, gos are 16 apart).
- **Go delay:** the go delayed by `OC_RD_START` = 23 starts bank 0's 16-row counter, so its read of row x is at G + 23 + x, one cycle before the deskewed psum row arrives (`OC_PSUM_LAT` = 24).
- **Bank 1** latches the pass fields when bank 0 reaches row S and reads S cycles later; its row index is a 4-bit counter that wraps every pass.
- **Decode** (per row x):
  - output row = `last` ∧ (x < m-size) (bank 1 also needs n-size > 8)
  - write-back = !`last`
  - `cX_first` = `first`
  - header = bank 0 output row 0
  - `out_ack` = `gemm_last` ∧ (x = m-size – 1) (on bank 1's timeline).
- **Output registers:** `cX_valid`, `c_hdr_valid` / `c_hdr_data` and `out_ack` one cycle after the sum stage, aligned with the C out registers.
- **Abort** clears the FIFO, go delay, counters and all stage registers.

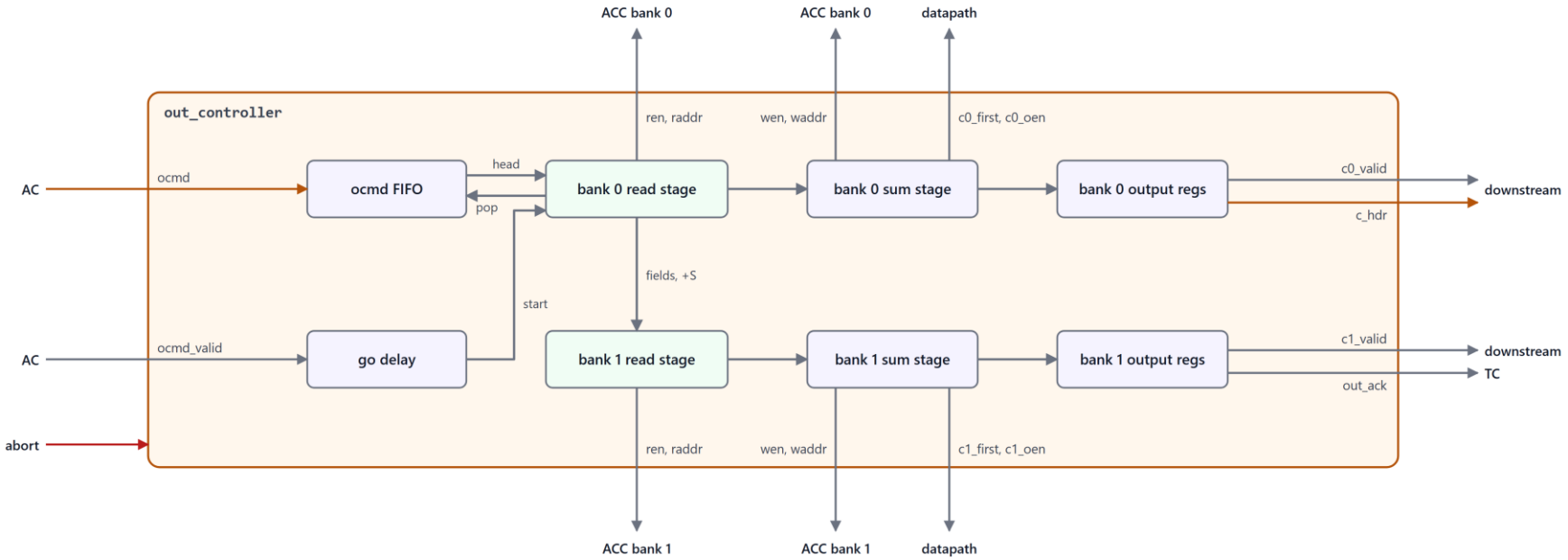


Figure 11: Output controller high level diagram

4.5 Accumulator datapath

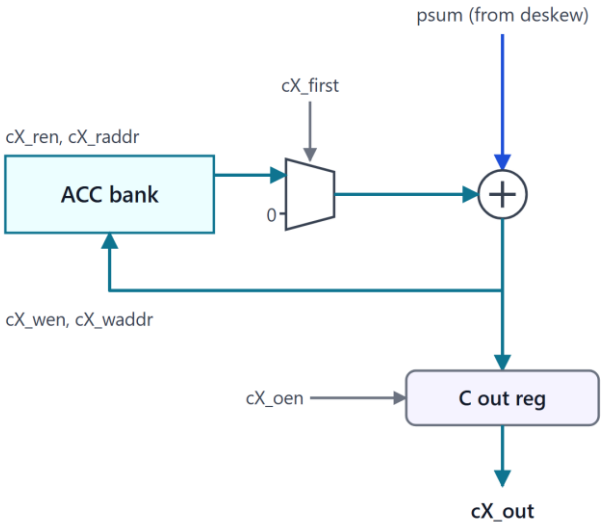


Figure 12: Accumulator datapath, one bank

- Per bank: sum = psum (sign-extended, 20 → 32 b) + (`cX_first` ? 0 : ACC read data).
- The sum is the ACC write data (`cX_wen`, non-last passes) and is loaded into the C out register on output rows (`cX_oen`).
- Row x is read at G + 23 + x and written at G + 24 + x; the next pass reads it 16 cycles later, so the read → add → write loop (2 cycles) never collides, and a read and write in one cycle are always to different rows.
- A block's first pass never uses the ACC data, so stale (or never-written, X) ACC contents never reach C.

4.6 Systolic array and PE

`systolic` (M = 16, NA = 8, NPIN = 20) is a 16 × 16 grid of `pe`. Activations enter the left edge (lane j = row j = K index j) and move right; weights and psums move down (lane i = column i = N index i); `a_first` travels with the activations and `w_latch` with the columns. Each PE computes `p_out = p_in + a × w` (registered) with its active weight.

Each PE double-buffers its weight with three flags: `flag` (active buffer), `upd` (buffer being loaded) and `lock` (loading stopped).

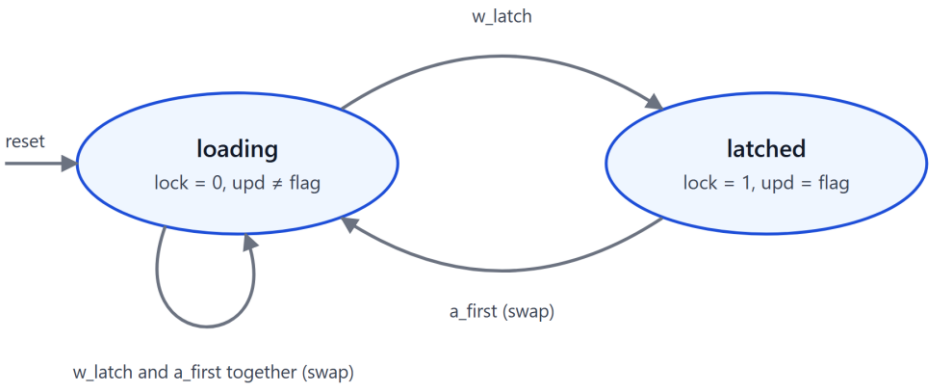


Figure 13: PE weight-buffer states

Table 20: PE weight-buffer states

State	Flags	Meaning
loading	lock = 0, upd != flag	The idle buffer shifts in the next tile while the active one computes
latched	lock = 1, upd = flag	The next tile is held; waiting for its pass's `a_first`

- `w_latch` stops the shift (loading → latched); `a_first` swaps to the new tile and resumes loading into the old one (latched → loading). At full rate both arrive together at PE[0][0] and the PE goes straight from loading to loading on the new tile.
- On abort: an extra `a_first` while loading, or a missing one while latched, leaves the buffers inconsistent for the next GEMM, hence the abort rules in Sec 2.7.4.

4.7 Skew, deskew and delay lines

- `delay_line` (D, W): a D-stage register chain of width W; D = 0 is a wire.
- `skew` (N, W, INCR, START_Z): N delay lines. With INCR = 1, lane j is delayed j (input skew); with INCR = 0, lane j is delayed N − 1 − j (deskew).
- Four input triangles (A and B, two halves each, 8 b) and two output triangles (20 b). The 8-cycle offset between halves comes from bank read timing, not from registers.

Their storage is listed in Sec 5.3.

4.8 Infrastructure

- `ram_1r1w` (EW, NE, DEPTH, RDW_X): one write and one read port, synchronous read (data one cycle after `re`, held while `re` is low), per-element write mask, no reset. With `RDW_X = 1` a read of the address being written returns X; bring-up ran every regression in both modes, so the design never depends on it.
- `sync_fifo` (W, DEPTH): flop-based, first-word fall-through; `full` / `empty` from registered state; `clr` flushes (used on abort).

5. Memories and storage

5.1 SRAM instances

Table 21: SRAM instances (all ram_1r1w: 1 read + 1 write port, 1-cycle synchronous read)

Instance	Purpose	Depth × width	Bits	Controller
`u_wei_buf0`, `u_wei_buf1`	B bank 0 / 1	32 × 64 b	2 × 2048	WC
`u_act_buf0`, `u_act_buf1`	A bank 0 / 1	32 × 64 b	2 × 2048	AC
`u_out_buf0`, `u_out_buf1`	ACC bank 0 / 1	16 × 256 b	2 × 4096	OC
Total			16 384 (2 KB)	

5.2 FIFOs

Table 22: Command and ID FIFOs (sync_fifo)

FIFO	Entries × width	Contents
TC pending	4 × 4 b	IDs of outstanding GEMMs
WC command	4 × 11 b	`wcmd_t`
AC command	4 × 26 b	`acmd_t`
OC command	4 × 22 b	`ocmd_t`

5.3 Flop arrays

The large flop arrays are listed below; controller state outside them (pointers, counters, flags, FIFO storage) is a few hundred flops in total.

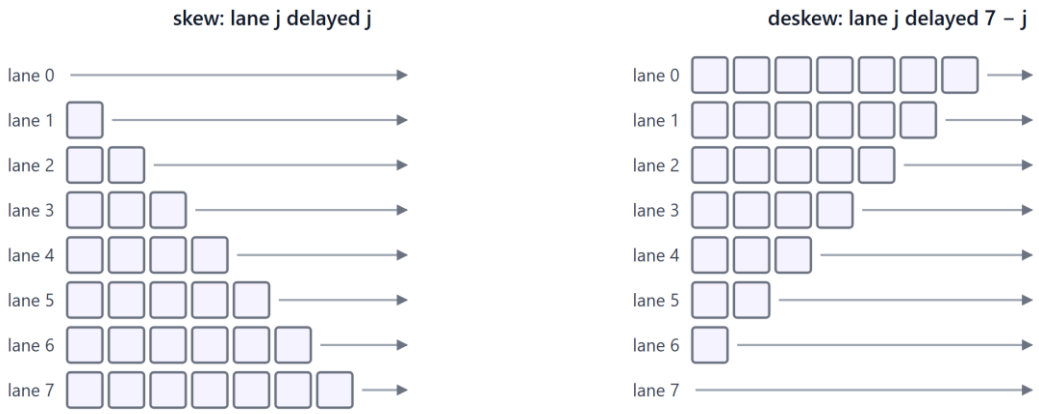


Figure 14: Skew and deskew triangles (one register per square, one lane wide)

Table 23: Large flop arrays

Array	Structure	Flops
A input skews	2 triangles × 28 regs × 8 b	448
B input skews	2 triangles × 28 regs × 8 b	448
Psum deskews	2 triangles × 28 regs × 20 b	1 120
C out registers	2 × 8 × 32 b	512
PE array	256 PEs × 49 (2 weights, activation, psum, flags)	12 544

Array	Structure	Flops
WC / AC / OC delay lines	WC: 2 × 8 (read enable, zero flag); AC: 8 + 16 (`a_first`); OC: 23 (go delay) + 8	≈ 80

6. Performance

Cycles from a descriptor's accept to its last C row, with the input streams ready:

cycles ≈ 16 · ceil(M/16) · ceil(N/16) · ceil(K/16) + 64

- **Fill, 32 cycles:** the first B tile's 16 rows arrive, then the 16-cycle weight load, before the first go.
- **Drain, 48 − 16 cycles:** the last pass's last `c1` row leaves 48 cycles after its go (array, deskew, registers, `c1` offset, 15 rows); its own 16 cycles are already counted.
- Back-to-back GEMMs pay fill and drain once per run.

Table 24: Measured on the RTL (16.00 cycles per pass in every case)

Workload	Passes	Cycles	Utilization
48×48 · 48×64	36	640	90.0%
4 × (48×48 · 48×64), back to back	144	2 368	97.3%
8 random shapes ≤ 100, back to back	240–348	3 887–5 630	98.8–98.9%

Latency: a pass's first C row leaves 25 cycles after its go (`c1`: 33); a GEMM's end response follows its last C row on bank 1's timeline.

7. Verification summary

- An end-to-end testbench drove `gemm_top` from descriptors and A/B streams (stream contract followed, padding lanes filled with garbage) and compared every C row, block header, `out_ack` and response with an exact reference GEMM, plus each pass's psums at the array and deskew outputs, the `a_first` diagonal, the pass-command fields and the WC/AC tile tags.
- Shapes from 1 to 100 per dimension and K = 65 536 with extreme values; random gaps on both streams; back-to-back GEMMs; `out_ack`/accept collisions; aborts swept over every pass phase with 0–4 GEMMs outstanding; every SRAM in both `RDW_X` modes. All runs pass.
- Deliberate faults (no dummy `a_first`, clearing `a_first` on abort, wrong output timing or fields, a wrong bank-1 mask) are caught.
- A file-driven testbench runs a real workload, a bank of 16 int8 3 × 3 filters over grayscale video frames via im2col (31 654 × 9 × 16 per frame): bit-exact on the RTL (three frames, 189 924 C rows) and on both routed netlists (2.0 ns and 1.5 ns) simulated with zero-delay cell models (one frame each, 63 308 C rows and 1 979 headers).

8. Implementation results

8.1 Synthesis

Pre-layout results for `gemm_top`: Yosys and OpenSTA through OpenROAD-flow-scripts (Docker image openroad/orfs, digest sha256:90683468be0f), Nangate45 typical corner (1.10 V, 25 °C). The scripts are in `syn/`. Every buffer is synthesized as flops; no SRAM macros are used. Timing uses an ideal clock and the library's default wire-load model, Sec 8.2 gives the routed results.

Table 25: Synthesis results (flat netlist, pre-layout)

Metric	Value
Cell area	422 645 µm² (0.42 mm²)
Cells / flops	184 921 / 32 114
Sequential share of area	34%

Metric	Value
Minimum clock period	1.705 ns (≈ 585 MHz); I/O paths need at most 1.107 ns
Peak throughput at that clock	256 MACs × 2 ops × 585 MHz ≈ 300 GOPS (int8)
Hold	Met (ideal clock)

Table 26: Cell area by block (hierarchical synthesis)

Block	Area (µm²)	Share
Systolic array: 256 PEs, ≈ 1 050 µm² each	268 950	60.8%
Accumulator banks: 2, 8 Kb of flops	72 918	16.5%
Weight banks: 2, 4 Kb of flops	35 698	8.1%
Activation banks: 2, 4 Kb of flops	35 178	8.0%
Skew and deskew triangles: 6	13 813	3.1%
gemm_top own logic	9 560	2.2%
TC, WC, AC, OC with their FIFOs	6 270	1.4%
Total	**442 386**	**100%**

Hierarchical synthesis keeps every module boundary, so its total is about 5% above the flat netlist; Table 26 gives the split and Table 25 the totals. The buffers, 16 Kb of flops, are a third of the area; SRAM macros would shrink them.

Critical path. From AC's `a_first[15]` register into the `p_out` register of PE (8, 7): the buffer tree for the `a_first` fanout (≈ 0.24 ns), the weight-select XOR and mux (≈ 0.18 ns; the PE selects its weight with `weight_flag_next = a_first ^ weight_flag`, Sec 4.6), the 8 × 8 multiply and the 20-bit psum add: 1.61 ns plus setup. Selecting the weight from a register would take about 0.4 ns off this path.

Expected paths. The combinational interlock (`weight_stall` → AC go → WC load start → weight bank read) arrives at 0.87 ns and the accumulate into the ACC banks at 0.96 ns; neither is close to critical.

Clock target. Synthesis maps with ABC's speed script, which does not use the clock period: a 1.6 ns target gives the same netlist (−0.105 ns slack). Place and route closes 1.5 ns (Sec 8.2).

8.2 Place and route

The same flow places and routes the flat netlist (`syn/run.sh pnr`) at 40% core utilization and 0.60 placement density, for two clock targets. Both route with no DRC violations and close timing.

Table 27: Place-and-route results, Nangate45 typical corner

Metric	2.0 ns target	1.5 ns target
Minimum clock period	1.66 ns (≈ 601 MHz)	1.50 ns (≈ 667 MHz)
Setup slack	+0.34 ns	0.00 ns (WNS = TNS = 0)
Hold slack	+0.03 ns	+0.02 ns
Cell area	428 256 µm²	430 999 µm²
Timing-repair buffers	3 983	6 967
Clock tree	7 330 buffers, 1 952 inverters; skew ≈ 0.04 ns	7 365 buffers, 1 824 inverters; skew ≈ 0.05 ns
Die	1.06 mm², 41% utilization (40% target)	1.06 mm², 41% utilization (40% target)
Routing	0 DRC violations	0 DRC violations
Worst IR drop	4.8 mV	6.3 mV

At 2.0 ns, gate sizing on the critical path recovers more than the wires and the clock skew cost, so the routed period is shorter than the pre-layout estimate. The 1.5 ns target closes too, for 0.6% more area and about 3 000 more repair buffers; repair stops at zero slack, so 1.5 ns is about this RTL's limit. In both, the critical path is `a_first` into a PE's `p_out`.

Corner. Timing is met at the typical corner (1.10 V, 25 °C) with 50 ps of clock uncertainty. The Nangate45 library in this flow ships no other corner, so the slow-corner setup and fast-corner hold analysis that signoff needs is not available.

Equivalence. The flow's equivalence check after clock-tree synthesis passed on the 2.0 ns run (netlists identical); the final check needs more than the 8 GB of memory available and was skipped. Gate-level simulations of both routed netlists stand in for it (Sec 7).

8.3 Power

Switching activity comes from RTL simulation. The testbench dumps a VCD over a steady-state window; each flop's toggle rate and duty are matched to the routed netlist by instance name (98% of the flops; OpenSTA propagates the rest and all combinational nets), and OpenSTA computes power on the routed design with its extracted parasitics: 500 MHz, Nangate45 typical corner.

Table 28: Power at 500 MHz by workload

Workload	Total	Array	Clock tree	Banks	Per useful MAC
Random int8 GEMMs: 4 × 48×48 · 48×64 back to back, 97% utilization	341 mW	173 mW	73 mW	60 mW	2.7 pJ
3 × 3 filter bank (16 filters) over a 240×135 grayscale video frame, im2col: K = 9, padding lanes zero	279 mW	120 mW	73 mW	59 mW	3.9 pJ
The same with random padding lanes	284 mW	124 mW	73 mW	59 mW	3.9 pJ
Idle: clock running, data static	139 mW		73 mW		

Table 29: Power by block, filter-bank workload (logic counted with the block of the flop it feeds)

Block	Power (mW)	Share
Systolic array	120.0	43.0%
Clock tree	72.7	26.0%
Accumulator banks	31.6	11.3%
Activation banks	14.5	5.2%
Weight banks	13.1	4.7%
Skew and deskew triangles	9.4	3.4%
Logic driving the output ports	9.3	3.3%
gemm_top own logic	5.7	2.0%
TC, WC, AC, OC	3.1	1.1%
Total	**279.4**	**100%**

Without the buffers. Counting each buffer's flops, the logic feeding them and their share of the clock tree (36.9 of 72.7 mW), the buffers draw 102, 99, 98 and 71 mW in the four rows of Table 28; the rest of the engine draws 239, 181, 185 and 68 mW: 1.9 pJ per MAC on random GEMMs and 2.5 pJ per useful MAC on the filter bank. SRAM macros would add their own read and write energy back.

Clocking dominates. The clock tree (73 mW) and the flops' clock-driven internal power (58 mW) are almost half of the filter-bank total and do not depend on the data: idle, the engine still draws 139 mW. v1 has no clock gating.

Real data toggles less. Image data switches about half as often as random int8 (0.10 against 0.18 transitions per flop per cycle) and cuts the array's power by 31%.

Padding lanes. Random instead of zero padding costs 4 mW (1.5%): the zero weights already keep the multipliers of those rows quiet, so zeroing the lanes in AC is not worth its logic.

Partial K tiles. Per hardware MAC the filter bank costs 2.2 pJ against random data's 2.7 pJ, but with K = 9, 7 of the 16 array rows do no useful work and still pay their clocking and pass psums down, so per useful MAC it costs 3.9 pJ.

PE registers. Each weight buffer changes value in 26% of cycles, so about 74% of its clocking could be gated; `p_out` and `a_out` change every cycle during compute.

Cross-check. Over 1 000 cycles of the filter-bank workload, a gate-level simulation of the routed netlist, read by OpenSTA directly (99% of pins annotated), gives 264 mW against 279 mW from the per-flop method: flop power agrees within 1%, but statistical propagation overestimates logic power by 19%, so the per-flop numbers above are about 6% high. The accumulator is never written in this workload and its flops stay X in the gate-level run; OpenSTA counts X flops as drawing no internal power, so their 30 mW of clocking is added back.

Limits. The simulation has no delays, so glitch power in the multipliers and adders is not counted. The banks are flop arrays here; as SRAM macros they, and about half of the clock tree (16k of the 32k flops), would go.

9. Limitations and future work

- **No mismatch detection** between upstream and the descriptor: a wrong stream gives a wrong C, and too few rows stall forever (no timeout). Possible later: debug markers on the streams.
- **Stale-beat contract** on abort; possible later: the descriptor ID on stream beats so stale beats are dropped in hardware.
- **No output backpressure.** A stalling downstream would need a local buffer; the array cannot stall mid-pass.
- **Fixed configuration:** int8 × int8 → int32, R = 16, one 16 × 16 array.
- **Power:** clock-gate the PE weight buffers (≈ 7–8 mW) and the whole array when no pass is in flight (up to ≈ 50 mW while waiting); remove the reset from datapath flops; banks as SRAM macros.
- **Clock speed:** take the weight select off the PE's multiply path (≈ 0.33 ns, for about 750 MHz); beyond about 800 MHz, a pipelined MAC.
- **v2:** the same array as four 8 × 8 cores with muxes at the core boundaries (independent, N-split, K-split modes), compared against v1 on area, timing and utilization.

Appendix A. Glossary

Term	Meaning
Pass	One B tile resident in the array while 16 rows of A stream through: 16 cycles
Block	One 16 × 16 tile of C, built over (K/16) passes
Go	The cycle AC starts a pass (G); every later step of the pass is a fixed offset from it
Load	The 16-cycle shift of a B tile into the PEs' idle weight buffers
Bank 0 / 1	Width halves of a buffer: lanes 0–7 / 8–15; bank 1 runs S = 8 cycles behind
DVS / DV	data / valid / stall handshake; data / valid only
k-size, m-size, n-size	Tile dimensions. A_tile is m × k, B_tile is k × n, and C_tile is m × n. 4 bits with 0 = 16. This is NOT the size of the entire A/B/C matrices in general, only of one tile used in computing one pass of the systolic array.